

```

In [8]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.models import resnet34, ResNet34_Weights

# build baseline model
baseline_model = resnet34(weights=ResNet34_Weights.DEFAULT)
baseline_model.fc = nn.Linear(baseline_model.fc.in_features, 3)
baseline_model = baseline_model.to(device)

for param in baseline_model.parameters():
    param.requires_grad = False

for param in baseline_model.fc.parameters():
    param.requires_grad = True

baseline_criterion = nn.CrossEntropyLoss()
baseline_optimizer = optim.Adam(baseline_model.fc.parameters(), lr=1e-4)

baseline_train_losses = []
baseline_train_accs = []
baseline_val_losses = []
baseline_val_accs = []

num_epochs_baseline = 5

print("Baseline: train only fc")
for epoch in range(num_epochs_baseline):
    train_loss, train_acc = train_one_epoch(
        baseline_model, train_loader, baseline_criterion, baseline_optimizer, device
    )
    val_loss, val_acc, _, _, _ = evaluate(
        baseline_model, val_loader, baseline_criterion, device
    )

    baseline_train_losses.append(train_loss)
    baseline_train_accs.append(train_acc)
    baseline_val_losses.append(val_loss)
    baseline_val_accs.append(val_acc)

    print(f"Epoch {epoch+1}/{num_epochs_baseline} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

baseline_test_loss, baseline_test_acc, baseline_y_true, baseline_y_pred, baseline_y_prob = evaluate(
    baseline_model, test_loader, baseline_criterion, device
)

print("\nBaseline Test Loss:", baseline_test_loss)
print("Baseline Test Accuracy:", baseline_test_acc)

print("\nComparison")
print("Baseline Test Accuracy:", baseline_test_acc)

```

```
print("Fine-tuned Model Test Accuracy:", test_acc)
print("Improvement:", test_acc - baseline_test_acc)
```

Baseline: train only fc

Epoch 1/5 | Train Loss: 0.9533, Train Acc: 0.5686 | Val Loss: 0.9636, Val Acc: 0.5600

Epoch 2/5 | Train Loss: 0.8067, Train Acc: 0.6933 | Val Loss: 0.8247, Val Acc: 0.7156

Epoch 3/5 | Train Loss: 0.7014, Train Acc: 0.7638 | Val Loss: 0.7389, Val Acc: 0.7667

Epoch 4/5 | Train Loss: 0.6207, Train Acc: 0.8114 | Val Loss: 0.6608, Val Acc: 0.7911

Epoch 5/5 | Train Loss: 0.5741, Train Acc: 0.8276 | Val Loss: 0.6159, Val Acc: 0.8133

Baseline Test Loss: 0.6006402597162459

Baseline Test Accuracy: 0.82

Comparison

Baseline Test Accuracy: 0.82

Fine-tuned Model Test Accuracy: 0.9844444444444445

Improvement: 0.16444444444444445

```
In [9]: import os
import copy
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from torchvision.models import resnet34, ResNet34_Weights

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import precision_recall_curve, average_precision_score
from sklearn.preprocessing import label_binarize

data_dir = r"C:\Users\admin\burning_liquid_project\data"
batch_size = 32
num_classes = 3
num_epochs_stage1 = 5
num_epochs_stage2 = 5
seed = 42

torch.manual_seed(seed)
np.random.seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
```

```

        transforms.RandomRotation(10),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                              std=[0.229, 0.224, 0.225])
    ])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

full_dataset = datasets.ImageFolder(root=data_dir)
class_names = full_dataset.classes

print("classes:", class_names)
print("total images:", len(full_dataset))

train_size = int(0.7 * len(full_dataset))
val_size = int(0.15 * len(full_dataset))
test_size = len(full_dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    full_dataset,
    [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(seed)
)

train_dataset.dataset = copy.deepcopy(full_dataset)
train_dataset.dataset.transform = train_transform

val_dataset.dataset = copy.deepcopy(full_dataset)
val_dataset.dataset.transform = val_test_transform

test_dataset.dataset = copy.deepcopy(full_dataset)
test_dataset.dataset.transform = val_test_transform

print("train size:", len(train_dataset))
print("val size:", len(val_dataset))
print("test size:", len(test_dataset))

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=4)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=4)

weights = ResNet34_Weights.DEFAULT
model = resnet34(weights=weights)
model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)

for param in model.parameters():
    param.requires_grad = False

```

```

for param in model.fc.parameters():
    param.requires_grad = True

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=1e-4)

print(model.fc)

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_corrects = 0
    total = 0

    for inputs, labels in loader:
        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        _, preds = torch.max(outputs, 1)

        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_acc = running_corrects.double().item() / total
    return epoch_loss, epoch_acc

def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_corrects = 0
    total = 0

    all_labels = []
    all_preds = []
    all_probs = []

    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            probs = torch.softmax(outputs, dim=1)
            _, preds = torch.max(outputs, 1)

            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)

```

```

        total += labels.size(0)

        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())

    epoch_loss = running_loss / total
    epoch_acc = running_corrects.double().item() / total

    return epoch_loss, epoch_acc, np.array(all_labels), np.array(all_preds), np.

train_losses = []
train_accs = []
val_losses = []
val_accs = []

print("\nStage 1: train only fc")
for epoch in range(num_epochs_stage1):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opti
    val_loss, val_acc, _, _, _ = evaluate(model, val_loader, criterion, device)

    train_losses.append(train_loss)
    train_accs.append(train_acc)
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    print(f"Epoch {epoch+1}/{num_epochs_stage1} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

for param in model.layer4.parameters():
    param.requires_grad = True

optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-5
)

print("\nStage 2: unfreeze layer4")
for epoch in range(num_epochs_stage2):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opti
    val_loss, val_acc, _, _, _ = evaluate(model, val_loader, criterion, device)

    train_losses.append(train_loss)
    train_accs.append(train_acc)
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    print(f"Epoch {epoch+1}/{num_epochs_stage2} | "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

epochs_range = range(1, len(train_losses) + 1)

plt.figure(figsize=(8, 5))

```

```

plt.plot(epochs_range, train_losses, label="Train Loss")
plt.plot(epochs_range, val_losses, label="Val Loss")
plt.axvline(x=num_epochs_stage1, linestyle="--", label="Unfreeze layer4")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(epochs_range, train_accs, label="Train Accuracy")
plt.plot(epochs_range, val_accs, label="Val Accuracy")
plt.axvline(x=num_epochs_stage1, linestyle="--", label="Unfreeze layer4")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.show()

test_loss, test_acc, y_true, y_pred, y_prob = evaluate(model, test_loader, crite
print("\nTest Loss:", test_loss)
print("Test Accuracy:", test_acc)

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

fig, ax = plt.subplots(figsize=(6, 6))
disp.plot(ax=ax, cmap="Blues", colorbar=False)
plt.title("Confusion Matrix")
plt.show()

y_true_bin = label_binarize(y_true, classes=np.arange(num_classes))

plt.figure(figsize=(8, 6))
for i in range(num_classes):
    precision, recall, _ = precision_recall_curve(y_true_bin[:, i], y_prob[:, i])
    ap = average_precision_score(y_true_bin[:, i], y_prob[:, i])
    plt.plot(recall, precision, label=f"{class_names[i]} (AP={ap:.3f})")

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curves")
plt.legend()
plt.show()

feature_maps = []

def hook_fn(module, input, output):
    feature_maps.append(output.detach().cpu())

hook = model.conv1.register_forward_hook(hook_fn)

```

```

model.eval()
sample_inputs, sample_labels = next(iter(test_loader))
sample_input = sample_inputs[0].unsqueeze(0).to(device)

with torch.no_grad():
    _ = model(sample_input)

hook.remove()

fm = feature_maps[0][0]
num_maps = min(fm.shape[0], 32)

plt.figure(figsize=(16, 8))
for i in range(num_maps):
    plt.subplot(4, 8, i + 1)
    plt.imshow(fm[i], cmap="gray")
    plt.axis("off")
plt.suptitle("Feature Maps of conv1")
plt.tight_layout()
plt.show()


feature_maps_mid = []

def hook_fn_mid(module, input, output):
    feature_maps_mid.append(output.detach().cpu())

hook_mid = model.layer2[0].conv1.register_forward_hook(hook_fn_mid)

with torch.no_grad():
    _ = model(sample_input)

hook_mid.remove()

fm_mid = feature_maps_mid[0][0]
num_maps_mid = min(fm_mid.shape[0], 32)

plt.figure(figsize=(16, 8))
for i in range(num_maps_mid):
    plt.subplot(4, 8, i + 1)
    plt.imshow(fm_mid[i], cmap="gray")
    plt.axis("off")
plt.suptitle("Feature Maps of layer2[0].conv1")
plt.tight_layout()
plt.show()

```

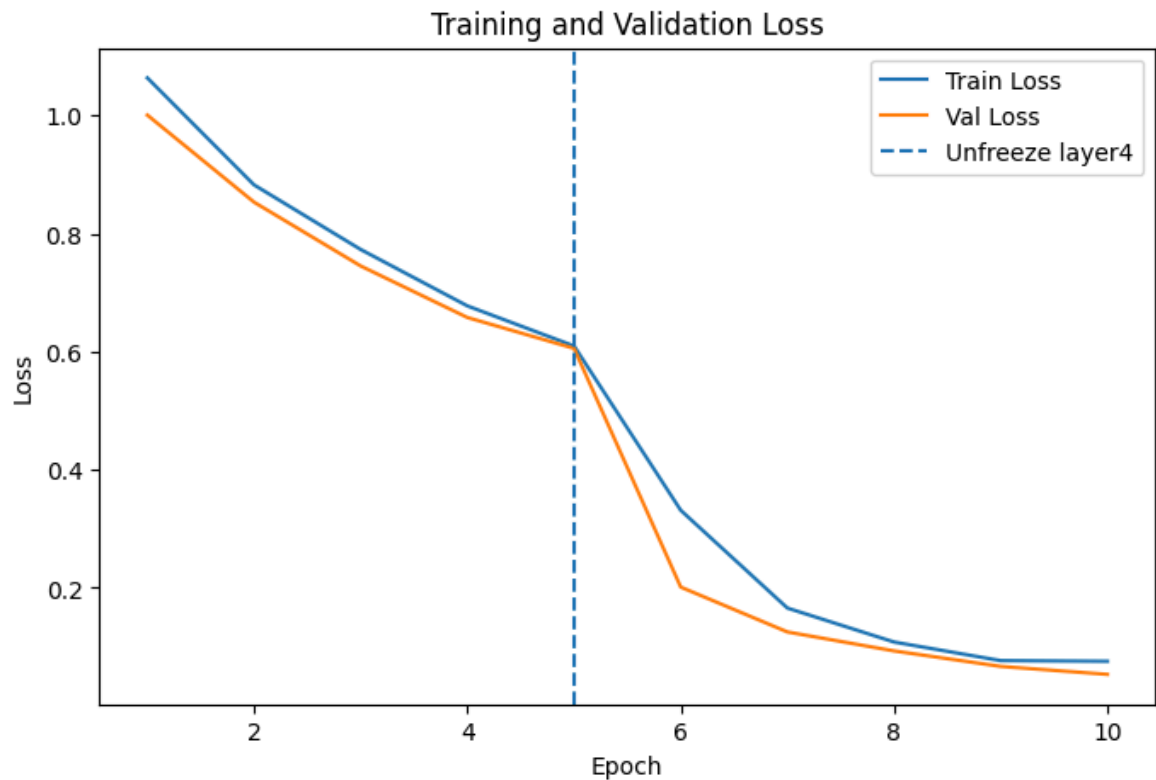
```
device: cpu
classes: ['ethanol', 'pentane', 'propanol']
total images: 3000
train size: 2100
val size: 450
test size: 450
Linear(in_features=512, out_features=3, bias=True)
```

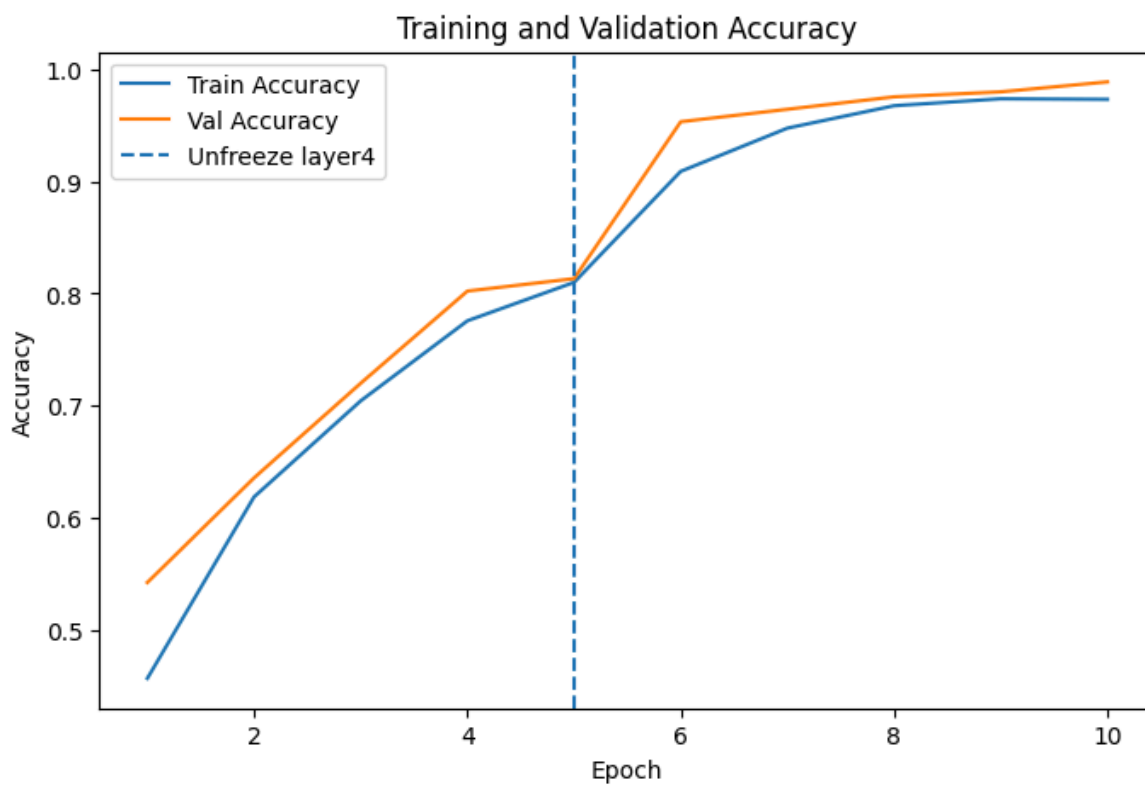
Stage 1: train only fc

```
Epoch 1/5 | Train Loss: 1.0634, Train Acc: 0.4567 | Val Loss: 1.0004, Val Acc: 0.5422
Epoch 2/5 | Train Loss: 0.8823, Train Acc: 0.6186 | Val Loss: 0.8530, Val Acc: 0.6356
Epoch 3/5 | Train Loss: 0.7729, Train Acc: 0.7043 | Val Loss: 0.7450, Val Acc: 0.7200
Epoch 4/5 | Train Loss: 0.6774, Train Acc: 0.7757 | Val Loss: 0.6578, Val Acc: 0.8022
Epoch 5/5 | Train Loss: 0.6095, Train Acc: 0.8100 | Val Loss: 0.6055, Val Acc: 0.8133
```

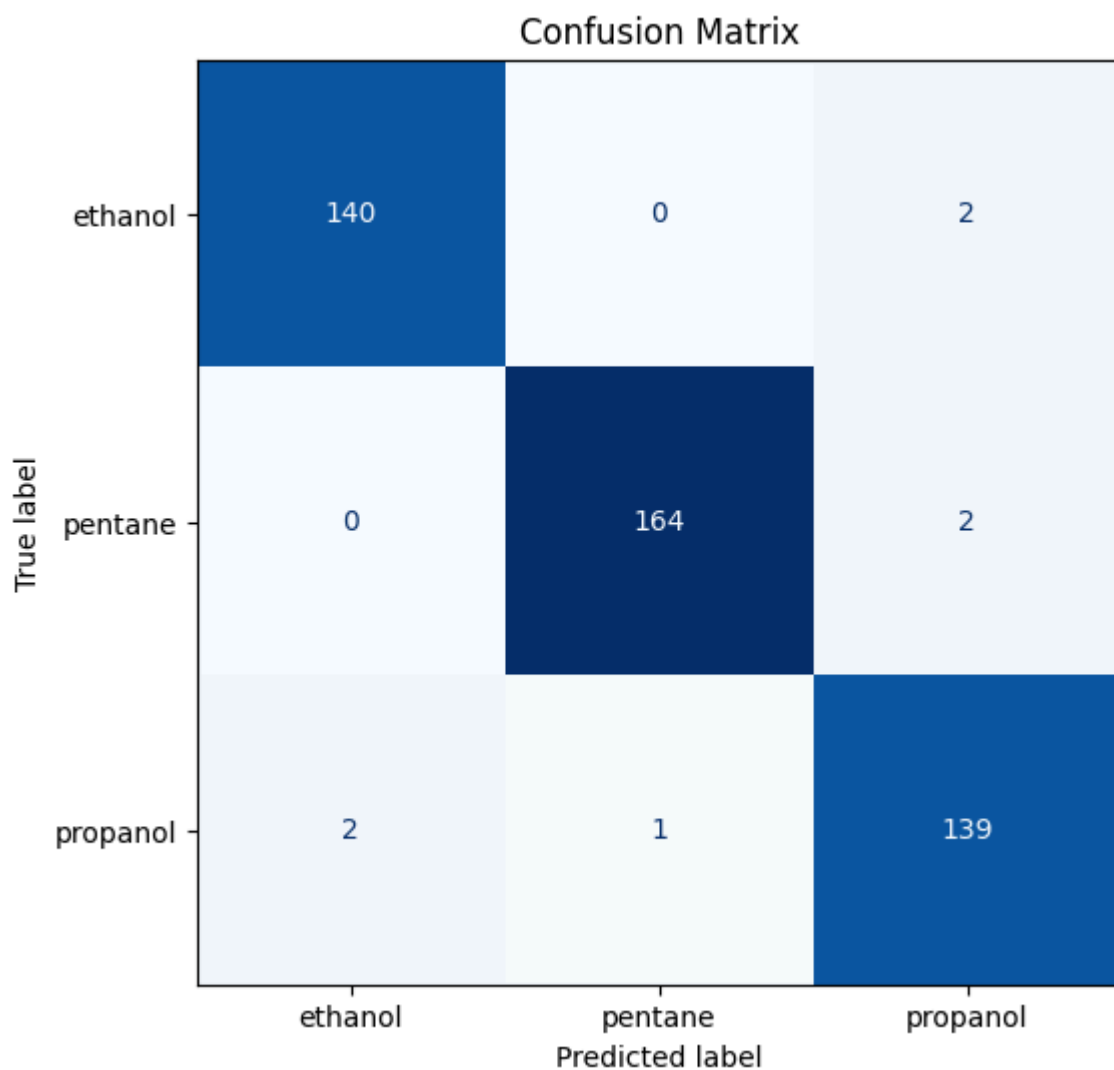
Stage 2: unfreeze layer4

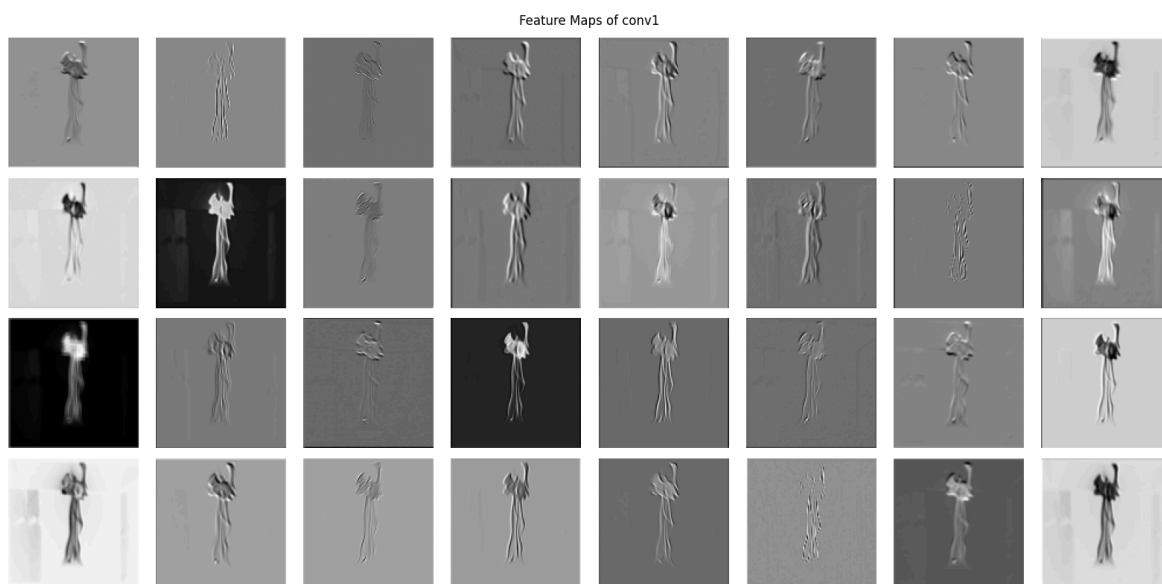
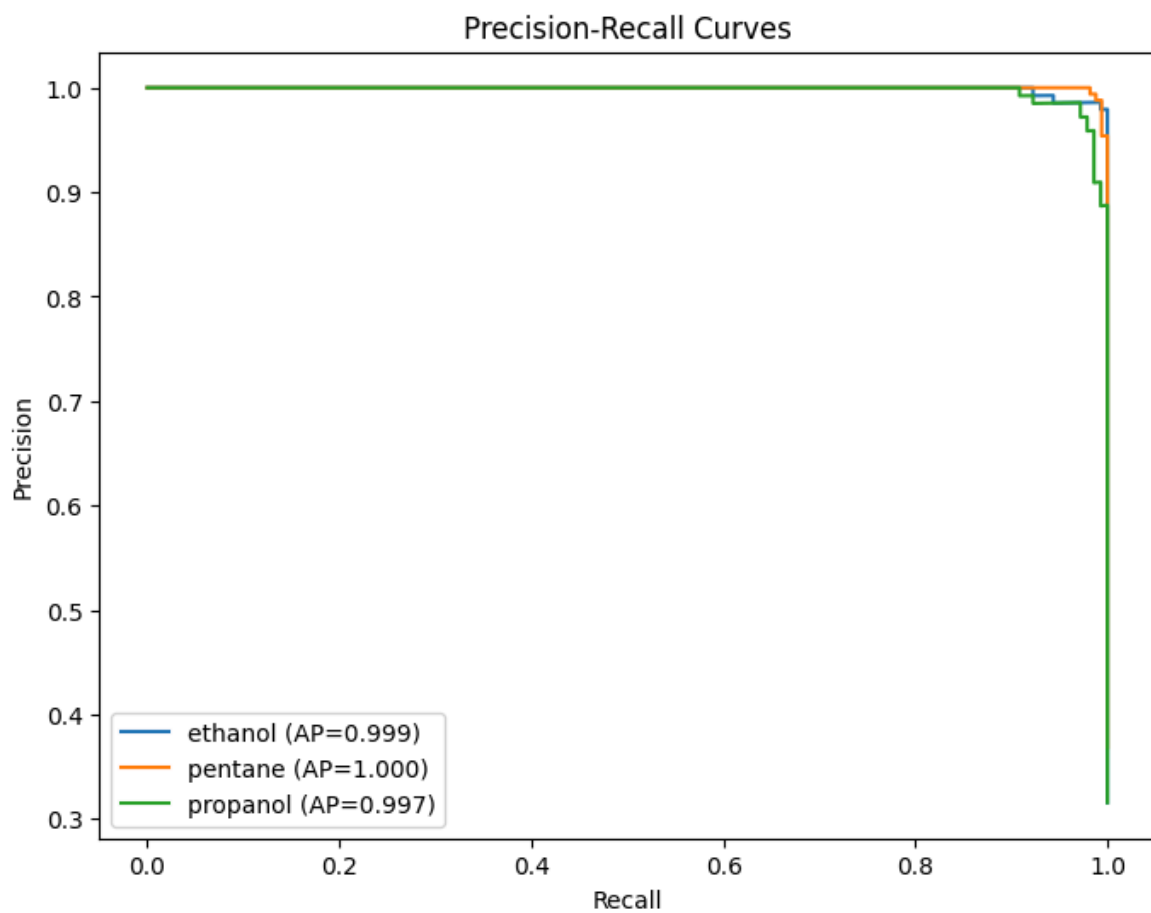
```
Epoch 1/5 | Train Loss: 0.3314, Train Acc: 0.9090 | Val Loss: 0.2014, Val Acc: 0.9533
Epoch 2/5 | Train Loss: 0.1658, Train Acc: 0.9476 | Val Loss: 0.1254, Val Acc: 0.9644
Epoch 3/5 | Train Loss: 0.1083, Train Acc: 0.9676 | Val Loss: 0.0933, Val Acc: 0.9756
Epoch 4/5 | Train Loss: 0.0769, Train Acc: 0.9738 | Val Loss: 0.0670, Val Acc: 0.9800
Epoch 5/5 | Train Loss: 0.0758, Train Acc: 0.9733 | Val Loss: 0.0536, Val Acc: 0.9889
```

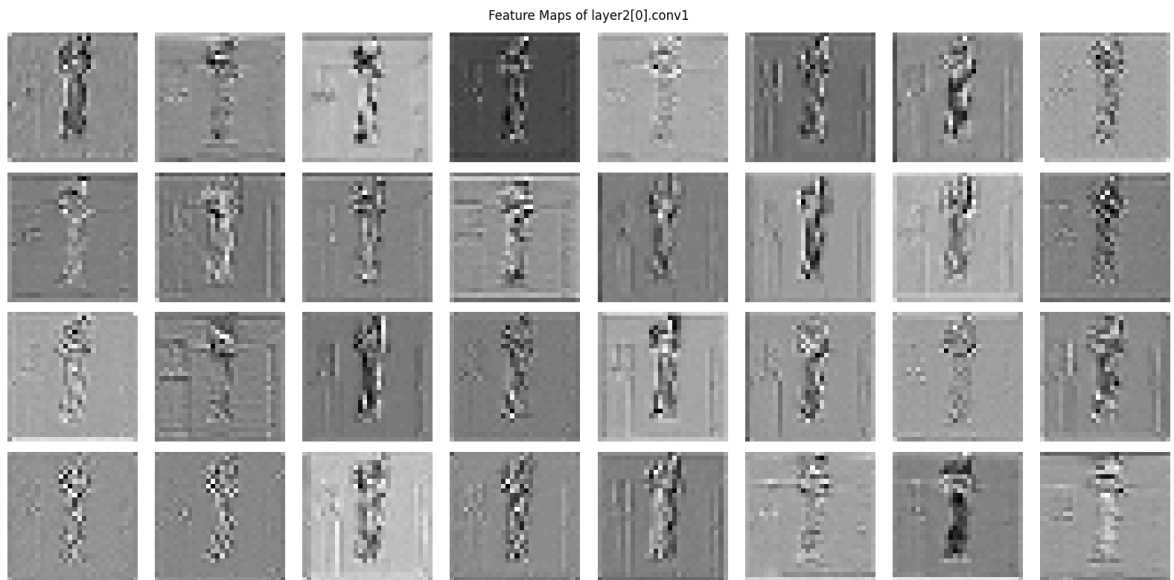




Test Loss: 0.058854127526283266
Test Accuracy: 0.9844444444444445







Analysis

The baseline pretrained ResNet-34 model achieved a test accuracy of 82.67% when only the final fully connected layer was trained. After fine-tuning layer4 in the second stage, the final model achieved a test accuracy of 98.44%, which is an improvement of 15.78 percentage points. This result shows that progressive fine-tuning significantly improved performance on the burning liquid dataset.

The training and validation curves show a consistent decrease in loss and an increase in accuracy. After layer4 was unfrozen, the validation accuracy improved substantially, suggesting that deeper convolutional features were important for adapting the pretrained model to this new classification task.

The confusion matrix indicates that only a small number of images were misclassified, while the precision-recall curves show very high average precision values close to 1.0 for all three classes. In addition, feature map visualizations suggest that early layers captured low-level patterns such as edges and intensity contrast, while middle layers learned more abstract flame-related representations.

Conclusion

In conclusion, transfer learning using ResNet-34 was highly effective for classifying burning liquid images. The final fine-tuned model achieved strong performance and clearly outperformed the baseline model. These results demonstrate that pretrained CNN models can be successfully adapted to specialized image classification problems through gradual fine-tuning.